

A fixed-state additive FWP network for streaming point-cloud recognition

Dian Yu*

University of New South Wales, Sydney, NSW 2052, Australia

Abstract

Most point-cloud networks obtain context by revisiting stored points, rebuilding neighborhoods, or applying a global maximum. These operations are effective offline but make a growing stream dependent on its history. PointProgrammerNet replaces point history with three fixed 128×170 matrices. For every observation, a normalized spatial address specifies where to write and a value vector specifies what to write; their outer product is added to each matrix. This simple construction makes independently encoded chunks exactly mergeable by matrix addition and also permits subtraction and exponential decay. Segmentation reads the matrices at a requested coordinate, while classification uses the matrices alone after all points have been discarded. We prove order invariance, chunk and distributed equivalence, fixed retained memory, and continuous dependence on incoming evidence. With XYZ input, the model obtains 92.95% point accuracy, 82.58% instance mIoU, and 79.10% category mIoU on ShapeNetPart, and 86.59% accuracy on ModelNet40. The three float32 states occupy 255 KiB regardless of stream length. At 8,192 accumulated points, updating one new chunk is $6.06\times$ faster than rebuilding the complete state, while the relative state error remains below 10^{-6} . The results show the accuracy cost and operational benefit of replacing an expanding point history with a bounded state that can be updated and combined directly.

Keywords: point clouds; streaming learning; additive state; fast-weight programmers; segmentation; classification

1 Introduction

A point cloud is an unordered set, but a sensor observes it over time. Processing within a frame should be independent of point order, and processing across frames should not require replaying the full history. PointNet [1] obtains a global descriptor by symmetric maximum pooling, while PointNet++ [2] builds multiscale metric neighborhoods. Both are effective offline. However, a maximum is not an additive state: fixed summaries cannot generally support deletion, independent chunk merging, and continuously weighted evidence by simple addition.

We study a stricter setting in which a point may be discarded immediately after it is written. Later computation accesses only fixed matrices, plus a query coordinate for segmentation. A learned key distributes each point across state rows, and a second-order value records mass, position, learned content, and cross moments. Summing these per-point writes produces a fixed-size state.

Independent chunks can therefore be merged, a stored subset can be subtracted, and old evidence can be decayed. New points update the state without a pass over earlier observations.

*Corresponding author. dian.yu2@student.unsw.edu.au, ORCID <https://orcid.org/0009-0002-0107-1014>

The same algebra supports distributed sensing: edge devices encode disjoint regions or time windows, transmit fixed matrices, and a central node adds those matrices before classification or segmentation. Segmentation reads a coordinate-conditioned field and stores no historical point features. Classification reads only state rows and discards the complete point list.

Our contributions are:

- a three-scale second-order fast-weight-programmer (FWP) state whose memory does not grow with the number of observed points;
- additive point writes that support exact streaming, hierarchical merging, and distributed aggregation without replaying chunk contents, and that admit a fully parallel write and read;
- a coordinate-conditioned segmentation head and a state-only classification head, neither of which pools over decoded points;
- proofs and experiments covering additivity, state operations, accuracy, and update cost, including a causal test showing that three parallel states outperform one state at an equal parameter budget.

2 Related Work

There are several established ways to reduce an unordered point set to a fixed summary. They differ mainly in what they give up at the moment a point is absorbed, and that choice decides which state operations remain available afterwards. We group the related work by that choice, then state which properties this architecture keeps.

2.1 Symmetric pooling and metric neighborhoods

PointNet applies pointwise transforms followed by maximum pooling [1], PointNet++ adds hierarchical metric neighborhoods [2], and Deep Sets characterizes permutation-invariant functions built from sums [3]. All three are invariant to point order, and the sum used by Deep Sets is additive in the strict sense of Equation (14). A growing stream nevertheless runs into two obstacles. A channelwise maximum discards the information needed to undo it, so a stored summary cannot be corrected once an observation is removed. A metric neighborhood is defined relative to the points currently held; insert or delete one point and the membership of other neighborhoods shifts with it, forcing the grouping to be rebuilt. Deep Sets escapes both, at the price of summing into a single global vector that carries no spatial address and cannot be evaluated at a requested coordinate. We keep the additive sum but spread it over 128 spatially addressed rows, so a coordinate-conditioned read remains possible without giving up additivity.

2.2 Fixed-size summaries over learned centers

VLAD compresses local descriptors by assigning them to learned centers and accumulating residuals [4]. NetVLAD makes that assignment soft and differentiable [5], and PointNetVLAD applies the construction to point clouds for place recognition [6]. The accumulator has the same shape as ours: a fixed matrix built by soft assignment to learned centers. It differs in what is accumulated and in what happens afterwards. The residuals are first order, whereas Equation (3) retains every moment up to second order, and the assignment is made in feature space, whereas our address is a normalized function of the coordinate and stays continuous in a query position. The decisive difference is the last step. NetVLAD intra-normalizes each block and then L_2 -normalizes the flattened descriptor, and neither operation is additive: normalizing two partial descriptors and adding them does not reproduce the descriptor of their union. The

raw accumulator is additive, but the published construction normalizes that property away and never uses it. We leave the accumulator unnormalized and move normalization to read time, preserving Proposition 1.

2.3 Attention over a fixed latent set

Set Transformer summarizes a set using inducing points and attention [7]. Perceiver IO maintains a latent array whose size is independent of the input and decodes arbitrary outputs, including dense per-element predictions, by attending to that latent with an output query array [8]. Reading a fixed latent with a query in order to produce dense output is their contribution, not ours. The difference lies on the encoding side. Softmax cross-attention normalizes over the observed elements, so the latent obtained from two disjoint chunks is not the sum of the two latents; the representation cannot be merged, subtracted, or decayed arithmetically, and no such property is claimed. Our write applies no normalizer over the observed set, so the state remains a sum of independent per-point terms. The meaning of a query also differs: Perceiver queries are learned position embeddings, whereas ours is a coordinate in the same space as the data, so one state can be evaluated at positions that were never observed.

2.4 Bounded recurrent state

Kernelized attention can be evaluated recurrently with a fixed-size state [9], and fast-weight programmers express such updates as additive outer products or delta corrections [10]. These are the mechanisms our write uses. Recent point-cloud work also adopts a bounded state through structured state space models; PointMamba serializes points with space-filling curves before the recurrence [11]. Serialization makes the hidden state a function of the traversal order, so it is not permutation invariant and two partial states cannot be added. We keep the bounded state but not the sequence: chunk and device equivalence rests on the sum being order free.

2.5 What this architecture refuses to do

Every family above trades one property for another: a maximum gives up invertibility, a metric neighborhood gives up locality of edits, a normalized descriptor gives up additivity, a softmax encoder gives up decomposability, and a serialized recurrence gives up order invariance. The design studied here follows from a single refusal at write time: nothing that couples one point’s write to another point may enter the persistent state. That excludes normalization over the observed set, comparison between points, selection, and ordering. Operations that aggregation methods usually perform while absorbing a point — density normalization, centring on a reference position, and nonlinear interpretation — are instead deferred to the read, where they no longer touch the algebra of the stored matrices. Table 1 states the resulting position.

What that refusal provides and what it costs are both measured rather than asserted. Section 4.2 isolates the contribution of the parallel-state structure, and Tables 3 and 4 place the resulting accuracy against hierarchical baselines.

We evaluate classification on ModelNet40 [12] and part segmentation on ShapeNetPart [13], which is derived from ShapeNet [14].

3 Method

Let $X = \{\mathbf{x}_i\}_{i=1}^N$, with normalized coordinates $\mathbf{x}_i \in \mathbb{R}^3$. PointProgrammerNet replaces the growing point history with three matrices $\mathbf{S}_s \in \mathbb{R}^{128 \times 170}$, where $s \in \{f, m, b\}$ denotes branches initialized at radii 0.08, 0.20, and 0.60. The radii, spatial centers, point encoders, and read

Table 1: Properties retained by families of set summaries. “Order-free write” means the summary does not depend on the order in which points are absorbed; “additive state” means the summary of a union equals the sum of the summaries; “bounded state” means retained memory does not grow with the number of observed points; “coordinate read” means the summary alone can be evaluated at a requested position.

Family	Order-free write	Additive state	Bounded state	Coordinate read
PointNet, maximum pooling	yes	no	yes	no
PointNet++, metric neighborhoods	yes	no	no	no
Deep Sets, global sum	yes	yes	yes	no
VLAD, NetVLAD, PointNetVLAD	yes	no ^a	yes	no
Set Transformer, inducing points	yes	no ^b	yes	no
Perceiver IO, output queries	yes	no ^b	yes	yes ^c
Linear attention, fast weights	yes ^d	yes	yes	no
State space models on serialized points	no	no	yes	no
PointProgrammerNet (this work)	yes	yes	yes	yes

^aThe raw accumulator is additive, but intra-normalization and L_2 normalization remove the property. ^bSoftmax over the observed elements couples them. ^cPerceiver IO decodes dense outputs from learned position queries, not from a coordinate in the input space. ^dThe outer-product update is itself order free, although these models are applied to ordered sequences.

networks are learned independently. The state is created for each object or stream and is separate from the trained model parameters.

All three branches have the same state dimensions and use the same equations. Their different initial radii only provide distinct starting conditions; the trained network decides what each branch represents. A branch first maps a coordinate to learned content with a two-layer point network,

$$\mathbf{g}_s(\mathbf{x}) = \text{ReLU}(\text{BN}(W_{s2} \text{ReLU}(\text{BN}(W_{s1}\mathbf{x} + \mathbf{b}_{s1})) + \mathbf{b}_{s2})) \in \mathbb{R}^{32}. \quad (1)$$

The matrices W_{s1}, W_{s2} and biases are learned separately in each branch, and both hidden layers have width 32. Consequently, equal state sizes do not imply repeated features. This nonlinear map is evaluated independently for each point. It therefore enriches what a point writes without coupling that write to any other point; additivity is preserved because only the subsequent outer products are accumulated in the persistent state.

3.1 Additive state construction

Each of the 128 state rows has a learned center $\mathbf{c}_{sj} \in \mathbb{R}^3$. These centers are not sampled from an input cloud. Before training, a Sobol sequence simply places 128 initial anchors across $[-0.9, 0.9]^3$ so that the rows begin with broad spatial coverage. Every branch receives its own trainable copy, and training may move the anchors through the bounded parameterization $\mathbf{c}_{sj} = 0.95 \tanh(\boldsymbol{\theta}_{sj})$. For one observed coordinate $\mathbf{x}$, the branch measures its distance to all 128 anchors and converts those distances into 128 continuous write fractions. The positive radius $r_s = \exp(-\lambda_s)$ controls how concentrated these fractions are. The normalized address is

$$k_{sj}(\mathbf{x}) = \frac{\exp[-\|\mathbf{x} - \mathbf{c}_{sj}\|^2/(2r_s^2)]}{\sum_{u=1}^{128} \exp[-\|\mathbf{x} - \mathbf{c}_{su}\|^2/(2r_s^2)]}. \quad (2)$$

The denominator fixes every point’s total write mass at one. This prevents dense chunks from changing the scale of an individual write and lets the radius control concentration rather than total magnitude. In practical terms, the centers act like 128 soft spatial bins. A point

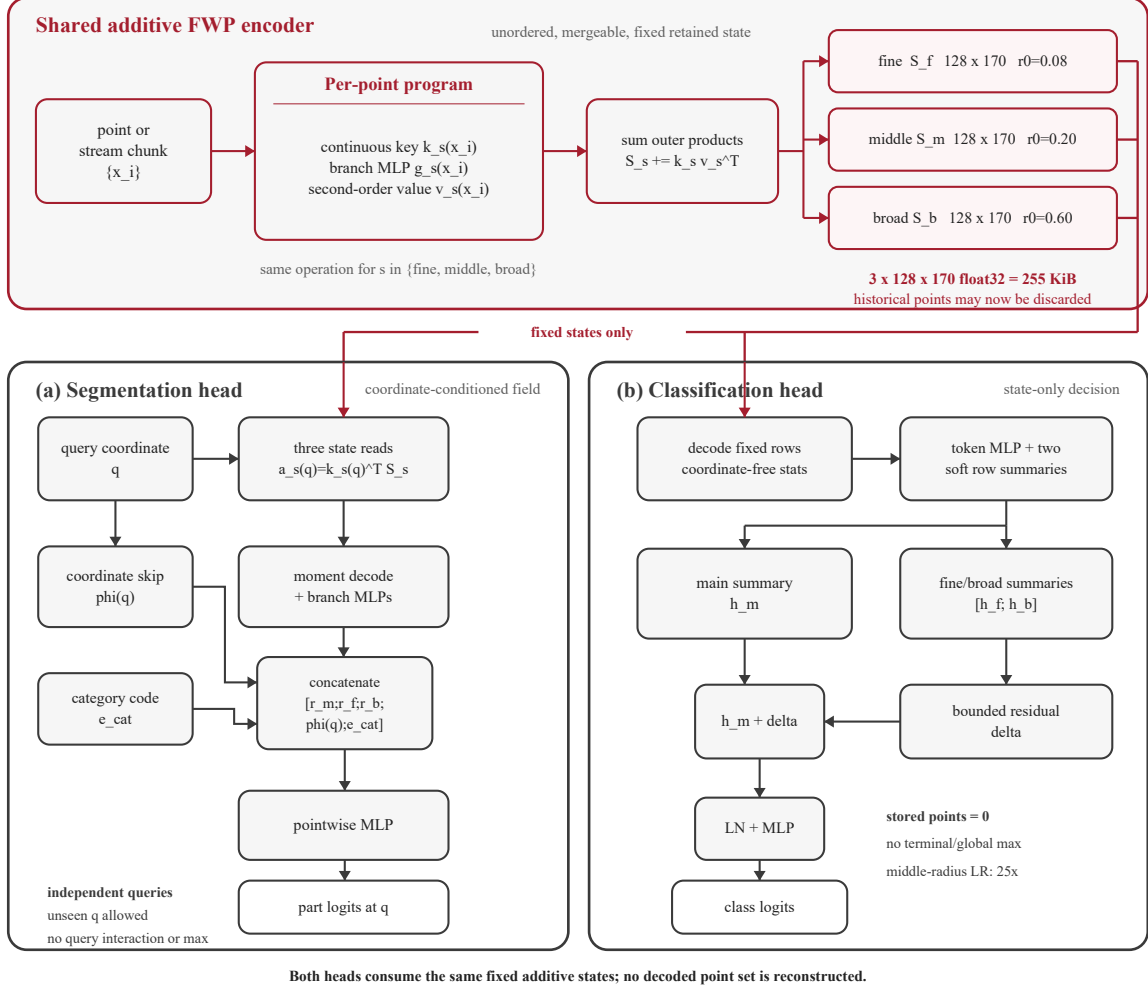


Figure 1: PointProgrammerNet architecture. Red paths carry the three equal-size additive FWP states after historical points have been discarded. The segmentation head reads each state at query coordinate q , then concatenates the decoded context, coordinate skip, and category code. The classification head decodes fixed rows and adds a bounded fine–broad residual to the middle-state summary. Neither head reconstructs or pools over a decoded point set.

near several centers gives those rows large fractions and all other rows small fractions; it is never assigned to only one bin. The address determines where the point contributes, while the following 170-dimensional value determines what statistics it contributes:

$$v_s(x) = [1, x, g_s, \text{vec}(x g_s^T), x^2, y^2, z^2, xy, xz, yz, g_s \odot g_s]. \quad (3)$$

The blocks have dimensions $1+3+32+96+6+32 = 170$ and retain mass, first moments, learned content, coordinate–content products, and second moments. The leading one later supplies the denominator for local averages. The six quadratic coordinate terms and 32 squared feature terms retain spread that cannot be recovered from a mean. Equivalently, the value vector collects every moment of (x, g_s) up to second order, with the feature–feature block restricted to its diagonal so that its width stays linear in the content dimension. A point changes state row j by

$$[\Delta S_s(x)]_{j,:} = k_{sj}(x) v_s(x)^T. \quad (4)$$

This rank-one update uses the defining FWP operation: the key selects where to write, the value supplies what to write, and independent writes superpose by addition [10]. PointProgrammer-

Table 2: The value written by one point. Every state row has these 170 columns.

Block	Dim.	Information retained
1	1	accumulated mass
$\mathbf{x}$	3	coordinate sum
$\mathbf{g}_s$	32	learned-content sum
$\mathbf{x}\mathbf{g}_s^\top$	96	position-content relation
$\mathbf{x}\mathbf{x}^\top$	6	geometric second moment
$\mathbf{g}_s \odot \mathbf{g}_s$	32	feature second moment

Net preserves this linear superposition, without a write normalization over the observed point set or a nonlinear overwrite of the previous state. Thus a row is a weighted accumulator rather than a slot for one selected point, and summing the outer products gives

$$\mathbf{S}_s(X) = \sum_{i=1}^N \mathbf{k}_s(\mathbf{x}_i) \mathbf{v}_s(\mathbf{x}_i)^\top = \mathbf{K}_s^\top \mathbf{V}_s. \quad (5)$$

No point is assigned to a single row and no point is compared with another point. Once its three outer products have been added, it may be released.

For a newly arrived chunk, $\mathbf{K}_s \in \mathbb{R}^{M \times 128}$ stacks its addresses and $\mathbf{V}_s \in \mathbb{R}^{M \times 170}$ stacks its values. The chunk update is one matrix product $\Delta \mathbf{S}_s = \mathbf{K}_s^\top \mathbf{V}_s$. The persistent state is updated by $\mathbf{S}_s \leftarrow \mathbf{S}_s + \Delta \mathbf{S}_s$, after which $\mathbf{K}_s$, $\mathbf{V}_s$, and the chunk points can be discarded. Chunk size changes temporary working memory, but not the retained representation.

The complete streaming workflow is therefore short. For each arriving chunk, the encoder (i) normalizes its coordinates in the shared reference frame, (ii) computes the three address and value matrices, (iii) adds the three products $\mathbf{K}_s^\top \mathbf{V}_s$ to the persistent states, and (iv) releases all chunk-dependent tensors. Independent devices perform the same four steps and transmit only their three matrices. At inference, the segmentation head evaluates only requested coordinates, whereas the classification head consumes the matrices directly. No later stage needs to regenerate a point list from the state.

3.2 Task readouts

For segmentation, $\mathbf{x}_i$ denotes an observed point that has already been added to the state and may then be discarded; $\mathbf{q}$ is only the coordinate at which a label is requested. On ShapeNetPart, queries use the labeled input coordinates, but the decoder receives the coordinate and fixed state rather than the stored point feature. A query produces an address by Equation (2) and reads

$$\mathbf{a}_s(\mathbf{q}) = \mathbf{k}_s(\mathbf{q})^\top \mathbf{S}_s = \sum_i \underbrace{\mathbf{k}_s(\mathbf{q})^\top \mathbf{k}_s(\mathbf{x}_i)}_{w_{si}(\mathbf{q})} \mathbf{v}_s(\mathbf{x}_i)^\top. \quad (6)$$

The coefficient $w_{si}(\mathbf{q})$ is an algebraic interpretation of the matrix read: it is large when point i wrote strongly to rows that $\mathbf{q}$ reads strongly. The implementation evaluates the left-hand matrix product directly; it neither reconstructs or revisits point i nor applies a global maximum over decoded points.

The first component of $\mathbf{a}_s(\mathbf{q})$ is the read mass $\mu_s(\mathbf{q})$. The implementation prevents unstable division in nearly empty regions with

$$\epsilon_s = 10^{-3} \text{mean}_{\mathbf{q}'} \mu_s(\mathbf{q}') + 10^{-20}, \quad \tilde{\mu}_s(\mathbf{q}) = \max(\mu_s(\mathbf{q}), \epsilon_s). \quad (7)$$

The mean is taken over the query coordinates of one sample and is detached from the gradient. The maximum in Equation (7) is only a scalar numerical floor; it neither compares point

features nor performs global maximum pooling. Dividing the stored coordinate, feature, cross-product, and square sums by $\tilde{\mu}_s$ yields local coordinate and feature means $\bar{\mathbf{x}}_s, \bar{\mathbf{g}}_s$, cross moment $E_{xg,s}$, geometric moment $E_{xx,s}$, and feature energy $E_{gg,s}$. They form the 170-dimensional descriptor

$$\mathbf{d}_s(\mathbf{q}) = [\bar{\mathbf{g}}_s, \bar{\mathbf{x}}_s - \mathbf{q}, \log \tilde{\mu}_s, \text{vec}(E_{xg,s} - \mathbf{q}\bar{\mathbf{g}}_s^\top), \text{vech}(E_{xx,s} - \bar{\mathbf{x}}_s\bar{\mathbf{x}}_s^\top), \sqrt{[E_{gg,s} - \bar{\mathbf{g}}_s \odot \bar{\mathbf{g}}_s]_+ + 10^{-8}}]. \quad (8)$$

Here vech keeps the six distinct entries of a symmetric 3×3 matrix, and $[\cdot]_+$ clips roundoff-induced negative variances. Its block dimensions are 32, 3, 1, 96, 6, 32, again totalling 170. The descriptor deliberately combines complementary statistics. The feature mean supplies local semantic content, the relative centroid and geometric covariance describe local position and shape, and the log mass records how much evidence supports the read. The coordinate–feature term preserves how content changes around the query, while the feature standard deviation exposes mixtures and boundary variation that a mean alone would hide. Each branch applies the same $170 \rightarrow 160 \rightarrow 160$ BatchNorm–ReLU decoder. The final pointwise head is

$$\mathbf{z}_{\text{seg}}(\mathbf{q}) = h_{\text{seg}}([\mathbf{r}_f(\mathbf{q}); \mathbf{r}_m(\mathbf{q}); \mathbf{r}_b(\mathbf{q}); \phi(\mathbf{q}); \mathbf{e}_{\text{cat}}]), \quad (9)$$

where $\mathbf{r}_s \in \mathbb{R}^{160}$ is a decoded state read, $\phi(\mathbf{q}) \in \mathbb{R}^{32}$ is a coordinate feature, and $\mathbf{e}_{\text{cat}} \in \mathbb{R}^{16}$ is the ShapeNetPart category code. The segmentation MLP is $528 \rightarrow 256 \rightarrow 128 \rightarrow 50$, with BatchNorm, ReLU, and dropout 0.3 after its first two affine layers. The coordinate path is a U-Net-like skip around state compression. The head acts on each query separately and never pools over decoded points. Historical features can be deleted; only coordinates that will later be labelled must remain. The same fixed state can also be queried at an unobserved coordinate.

For classification, no coordinates are retained. For row j , the 170 stored entries are partitioned as

$$[\mu_{sj}, \mathbf{p}_{sj}, \mathbf{f}_{sj}, \text{vec}(\mathbf{C}_{sj}), \text{vech}(\mathbf{Q}_{sj}), \mathbf{u}_{sj}],$$

that is, mass, coordinate sum, feature sum, coordinate–feature sum, symmetric coordinate second moment, and squared-feature sum. Row normalization uses

$$\tilde{\mu}_{sj} = \max\left(\mu_{sj}, 10^{-4} R^{-1} \sum_{u=1}^R |\mu_{su}| + 10^{-20}\right), \quad (10)$$

whose lower bound is detached during back-propagation. With $\bar{\mu}_s = R^{-1} \sum_u \tilde{\mu}_{su}$, division by $\tilde{\mu}_{sj}$ produces

$$\mathbf{d}_{sj}^{\text{row}} = [\bar{\mathbf{g}}_{sj}, \bar{\mathbf{x}}_{sj}, \log(\tilde{\mu}_{sj}/\bar{\mu}_s), \text{vec}(\text{Cov}_{xx,sj}), \text{Std}_{g,sj}, \text{vec}(\text{Cov}_{xg,sj})] \in \mathbb{R}^{170}, \quad (11)$$

with block dimensions 32, 3, 1, 6, 32, 96. An independent $170 \rightarrow 128 \rightarrow 128$ LayerNorm–GELU network encodes every row, giving $\mathbf{T}_s \in \mathbb{R}^{128 \times 128}$. Two learned score columns summarize the fixed rows:

$$\mathbf{U}_s = \text{softmax}_{\text{rows}}(\text{score}_s(\mathbf{T}_s)), \quad \mathbf{h}_s = H_s(\text{vec}(\mathbf{U}_s^\top \mathbf{T}_s)) \in \mathbb{R}^{256}. \quad (12)$$

This softmax is over 128 retained state rows, not over the input points, so its cost is fixed after encoding. The middle summary is the reference and the other two form a bounded residual:

$$\begin{aligned} \boldsymbol{\delta} &= \sigma(a) \tanh(W[\mathbf{h}_f; \mathbf{h}_b] + \mathbf{b}), \\ \mathbf{z}_{\text{cls}} &= h_{\text{cls}}(\text{LN}(\mathbf{h}_m + \boldsymbol{\delta})). \end{aligned} \quad (13)$$

The residual projection starts at zero and $\sigma(a)$ starts at 0.1. The classifier h_{cls} is $256 \rightarrow 256 \rightarrow 40$, with GELU and dropout 0.3. All radii are learned; only the middle radius uses a $25 \times$

learning-rate multiplier and zero weight decay. The classifier uses only the fixed matrices and has no terminal point maximum. Two score columns provide complementary fixed-cost views of each state instead of collapsing every row through one average. Using the middle branch as an initially stable reference and admitting the fine and broad summaries through a zero-started bounded residual prevents the three branches from perturbing one another before each has learned a useful summary; the residual can then grow when the auxiliary evidence improves classification.

3.3 Streaming properties

For arbitrary point multisets A and B , Equation (5) gives

$$\mathbf{S}_s(A \uplus B) = \mathbf{S}_s(A) + \mathbf{S}_s(B). \quad (14)$$

Finite sums also make the state invariant to point order. Thus separately encoded chunks, devices, or branches of an aggregation tree can be merged by addition. If a removable subset state is available, it can be subtracted; exponential forgetting uses $\mathbf{S}_{s,t} = \rho \mathbf{S}_{s,t-1} + \mathbf{S}_s(X_t)$. Exact distributed equivalence requires common trained weights, coordinate normalization, and reference frame.

Let $F_s(\mathbf{x}) = \mathbf{k}_s(\mathbf{x})\mathbf{v}_s(\mathbf{x})^\top$ be one point's rank-one FWP write, so that $\mathbf{S}_s(X) = \sum_{\mathbf{x} \in X} F_s(\mathbf{x})$.

Proposition 1 (Order, chunks, and devices). *For point multisets A, B , $\mathbf{S}_s(A \uplus B) = \mathbf{S}_s(A) + \mathbf{S}_s(B)$, and the state is invariant to point order. Devices using identical trained encoders, coordinate normalization, and reference frames therefore obtain the centralized state by adding their independently encoded states.*

Proof. Both sides expand into the same finite collection of per-point matrices $F_s(\mathbf{x})$. Reordering or regrouping a finite sum does not change it in real arithmetic. Float32 results may differ only through accumulation order. $\square$

To make the distributed claim explicit, suppose device d observes point multiset $X^{(d)}$ and all devices use the same encoder. A central node forms

$$\mathbf{S}_s^{\text{global}} = \sum_{d=1}^D \mathbf{S}_s(X^{(d)}) = \mathbf{S}_s\left(\biguplus_{d=1}^D X^{(d)}\right). \quad (15)$$

The equality follows by expanding both sides into the same sum of per-point outer products. It holds whether the partial states arrive simultaneously, asynchronously, or through a tree of intermediate aggregators. Classification then uses only the merged matrices. Segmentation additionally receives the coordinates to be queried and the object category code. Raw point features need not leave the sensing devices.

The same algebra gives reversible state operations when the required partial state is available. If Y is a known subset, then $\mathbf{S}_s(X \setminus Y) = \mathbf{S}_s(X) - \mathbf{S}_s(Y)$. This removes the aggregate contribution of Y without replaying X . Multiplication by $\rho \in [0, 1]$ before an append implements exponential forgetting. These statements concern the representation itself; subtraction cannot recover individual points from a state when their separate subset state was not retained.

Proposition 2 (Retained memory and update cost). *With R_s rows and D_s columns per branch, retained memory is $O(\sum_s R_s D_s)$, independent of history length. Encoding M new points costs $O(M \sum_s R_s D_s)$ and does not revisit earlier observations.*

Proof. Only the fixed matrices persist. A new chunk contributes new outer products of the same dimensions regardless of the number of earlier chunks. $\square$

